(Horner's Rule)

Hashing Intro for Tutorial

1) Need for Instant Search

Data needs to be accessed fast
Slow: linear scan [O(n)]
Fast: Hashing [O(1)]

2) Hashing

↓

Hash tables = Arrays but! Not in order
INSTEAD → Deterministic mechanism
"Librarian knows all places."

α = load factor
m = total spaces in table
n = number of items in table.

3 Horners Rule

A simple, good (Not the best) solution to creating a signature.
Treat the input as a polynomial and sum.

↓

Normal computation of polynomial sum is slow! $O(n^2)$

↓

Horners Rule → Solve from right to left. Now each step is one multiplication and one addition.

We the [illegible]

We then can add a $2m+1$ to ensure all keys fit in table.

4) When Hashes Collide

Open Hashing: Add a prime / look next one
Chained Hashing: Each slot becomes a head in a linked list. Add collisions to chain

5) Load Factors Role

$\alpha = \frac{n}{m}$

Open Hashing

α (load fac)	Avg search length
0.5	2.0
0.9	10.0
0.99	100.0

Search length

$$\frac{1}{(1-\alpha)}$$

Chained Hashing

α (load fac)	Avg search length
0.5	1.25
0.9	1.5
0.99	2.0

Search length

$1 + \alpha/2$

6) Prime Numbers (Even More optimisation)

Combining Modular with a prime m spread data better → less collisions.

e.g: m = 137